

ESP32 ↔ Rhino AMR Communication

How the pieces connect, in plain language — before finalizing the BOM

This explainer answers, in order: whether you need a separate battery for the ESP32, what Wi-Fi and MQTT actually are and how they relate to each other, and the full in-between process that happens when you press a key on your laptop and the AMR moves. The BOM should only be finalized after this architecture is settled — the parts list depends directly on which of the two stages below you're building.

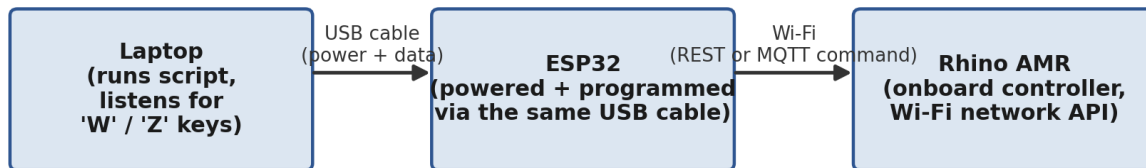
1. Do you need a separate battery for the ESP32?

Short answer: **it depends on whether the ESP32 stays physically cabled to your laptop or not.** There are two distinct stages here, and they have two different answers.

Stage 1 — Bench testing (what you're describing right now)

If the ESP32 sits on your desk connected to the laptop by a USB cable while you test 'W' and 'Z' key presses, then **no, you do not need a separate battery.** A USB cable carries both power and data on the same wire. The laptop's USB port supplies the ESP32's 5V, and the same cable can carry your keypress commands to it as plain serial text. This is the simplest possible setup and it's exactly where you should start.

Stage 1 — Bench Testing (ESP32 tethered to laptop, no separate battery)



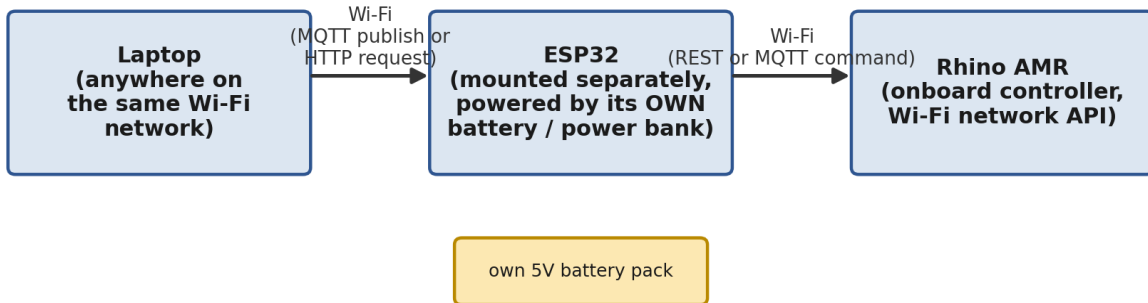
No separate battery needed here — the laptop's USB port powers the ESP32.
Only the ESP32→AMR leg needs wireless, because that's a separate physical machine.

Stage 2 — Deployed on/near the AMR (later, once bench testing works)

Once the ESP32 needs to be mounted on or near the forklift itself — no longer tethered to your laptop by a wire — **yes, it now needs its own independent power source** (a small battery pack or power bank), because there is no cable left to power it from anything. This was the earlier decision too: it should not draw from the AMR's 48V bus, and it can no longer draw from your laptop's USB port either once the cable is gone. So it needs its own supply.

Stage 2 — Deployed (ESP32 mounted separately, own battery required)

Both hops are now wireless — no cable to the laptop at all, so the ESP32 must supply its own power.



Practical suggestion: build and prove Stage 1 first (USB-tethered, no battery, easy to debug). Only add a battery to the BOM once you're actually ready to unplug the ESP32 from the laptop.

2. What is Wi-Fi?

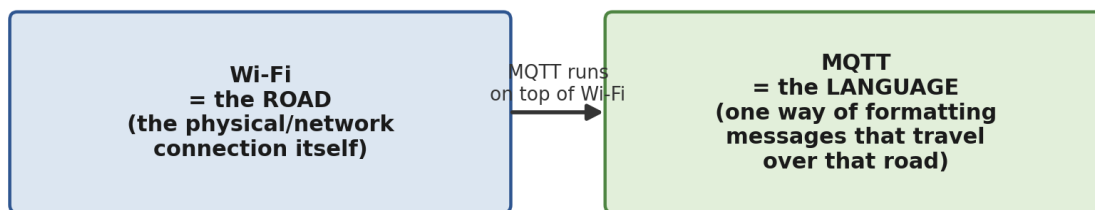
Wi-Fi is simply a wireless version of an Ethernet cable — a way for two devices to exchange data without a physical wire between them, using radio waves. It does not by itself define *what* the data means; it just gets bytes from one device to another over the air, the same way a road gets a truck from one city to another without caring what's inside the truck.

For your ESP32, having Wi-Fi means it can join the same wireless network the Rhino AMR's controller is on, and once it's on that network, it can send and receive messages to the AMR's IP address — the same way your laptop can talk to any website once it's connected to a router.

3. What is MQTT — and how is it different from Wi-Fi?

This is the part that's usually confusing, so here's the key idea: **Wi-Fi and MQTT are not two alternative ways to connect** — **MQTT runs ON TOP of Wi-Fi (or Ethernet)**. They solve different problems, not the same problem.

Wi-Fi vs MQTT — they are not alternatives to each other



MQTT is a lightweight messaging format built for small devices like the ESP32. Instead of each device calling another directly, everyone connects to a central relay called a **broker**. A device that wants to send data **publishes** a message to a named channel called a **topic** (e.g. rhino/forklift01/command). Any device that wants that data has **subscribed** to that same topic, and the broker delivers the message to it automatically. Nobody talks to anybody directly — everything passes through the broker.

	Wi-Fi	MQTT
What it is	The wireless connection itself	A message format that rides on top of a connection
Analogy	The road	The delivery method — packages with labeled addresses
Needed for	Any wireless communication at all	Only if you choose MQTT as the message style (REST/HTTP is the alternative)

Whether you actually need MQTT depends entirely on what Reeman's AMR controller expects. If their API is plain REST (simple HTTP requests, like the examples used earlier in this project), you don't need MQTT at all — you'd just send an HTTP request over Wi-Fi. If Reeman's system is MQTT-based, then your ESP32 needs an MQTT client library and the broker's address/topic names from them.

4. The full in-between process: pressing 'W' to move the AMR forward

Here is exactly what happens, step by step, for the bench-testing setup (Stage 1 above). This is the version to build and prove first.

- 1 You press the 'W' key while your laptop script is running and listening for keyboard input.
- 2 The laptop script (a short Python script is the usual choice) detects the keypress and sends a short text command — e.g. the word FORWARD — down the USB cable to the ESP32, using serial communication.
- 3 The ESP32 receives that text over its serial (UART) connection — this part has nothing to do with Wi-Fi yet; it's just a wired data line, same idea as a keyboard-to-computer cable.
- 4 The ESP32's code interprets FORWARD and now needs to tell the actual Rhino AMR to move. This is where Wi-Fi comes in: the ESP32 uses its built-in Wi-Fi radio to send a message to the AMR's onboard controller, over whatever protocol Reeman's system expects (a REST request or an MQTT publish).
- 5 The Rhino AMR's onboard computer receives that instruction on its network interface, and its motion-control layer drives the traction motor forward.
- 6 (Optional) The AMR can report status back the same way — battery level, current state, whether the command succeeded — which the ESP32 can pass back up to your laptop over the same USB serial line, so you see confirmation on screen.

Once this works reliably on the bench, the only thing that changes for deployment (Stage 2) is step 1–3: instead of a USB cable, the laptop sends the same kind of command wirelessly (Wi-Fi, using MQTT or HTTP) to the now-battery-powered ESP32. Steps 4–6 stay exactly the same.

5. What this means for your BOM

Now that the architecture is clear, here's how it maps to parts:

- **Stage 1 (bench):** ESP32 board, one USB cable (power + programming + data — no separate battery, no Wi-Fi hardware purchase needed since it's built into the ESP32).
- **Stage 2 (deployed):** same ESP32, plus an independent 5V battery pack/power bank, plus whatever mounting/enclosure hardware suits where it sits on or near the AMR.
- **Either stage:** you do not need any MQTT-specific hardware — MQTT (if Reeman's system uses it) is just software running on the same Wi-Fi radio already built into the ESP32. No extra part required for it.
- The one open item that still needs an answer from Reeman before the deployed-stage BOM is final: whether their AMR controller speaks REST or MQTT, and the connection details (IP/broker address, credentials, topic names or endpoint paths).